

B. M. S. COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to VTU, Belagavi

Department of Machine Learning

Program: B.E. in Artificial Intelligence and Machine Learning

Academic Year: 2025-26

Python Assignments and Exercises Report

Student Name:	CHITTARA D
USN:	1BM24AI038
Semester Section:	& 5E

ABSTRACT

This report presents the Python assignments, 41 exercises, and six mini projects completed as part of the ECRIIO assessment. The exercises provide the foundation in Python programming, file handling, object-oriented programming, data analysis, Flask, and MQTT. The mini projects apply these concepts to practical problems including a calculator, transaction management, Titanic dataset analysis, feature engineering, a To-Do web application, and MQTT-based communication. The report focuses primarily on the mini projects while providing a concise summary of the exercise component.

TABLE OF CONTENTS

INDEX	DESCRIPTION	PAGE
A	Python Exercises – Summary	3
1	Calculator	5
2	Transaction Management System	7
3	Titanic Dataset Analysis	9
4	Data Analysis & Feature Engineering	11
5	Simple To-Do Web Application	13
6	MQTT Publisher & Subscriber	15
	Overall Conclusion	16

ABSTRACT

This report presents the Python assignments, 41 exercises, and six mini projects completed as part of the ECRIIO assessment. The exercises provide the foundation in Python programming, file handling, object-oriented programming, data analysis, Flask, and MQTT. The mini projects apply these concepts to practical problems including a calculator, transaction management, Titanic dataset analysis, feature engineering, a To-Do web application, and MQTT-based communication. The report focuses primarily on the mini projects while providing a concise summary of the exercise component.

PART A — PYTHON EXERCISES

A.1 Exercise Overview

The ECRIO exercise component contains 41 exercises and is kept concise in this report because the main focus is on the mini projects. The exercises progress from Python fundamentals and file processing to object-oriented programming, data analysis, Flask, and MQTT. They provided the concepts that were later applied more practically in the mini projects.

A.2 Exercise Groups

- Exercises 1–6: Python fundamentals, functions, control flow, file handling, regular expressions and encoding.
- Exercises 7–12: Classes, inheritance, encapsulation, logging, collections, exceptions and assertions.
- Exercises 13–21: NumPy, Pandas, cleaning, grouping, normalization, pivot tables and trend analysis.
- Exercises 22–30: Flask routes, templates, forms, sessions and JSON handling.
- Exercises 31–41: MQTT publisher/subscriber communication, topics, QoS and IoT concepts.

A.3 Common Tools and Concepts

Python, pathlib, regular expressions, CSV, JSON, NumPy, Pandas, Flask, Paho-MQTT, exception handling, logging and basic web/IoT concepts.

A.4 Exercise Flow

LEARN CONCEPT

↓

IMPLEMENT EXAMPLE

↓

TEST / VALIDATE

↓

OBSERVE OUTPUT

↓

APPLY LEARNING

PART A — PYTHON EXERCISES

A.5 Learning

- Built confidence with Python syntax, functions, loops and data structures.
- Practiced reading, writing and transforming different forms of data.
- Understood how OOP concepts improve program structure.
- Gained experience with data cleaning and analysis using Python libraries.
- Explored how Python can be extended into web and IoT applications.

A.6 Conclusion

The 41 exercises served as the technical foundation for the later mini projects. Instead of treating the exercises as isolated tasks, they helped build a progression from basic programming to application-oriented Python development.

Mini Project 1: Menu-Driven Calculator

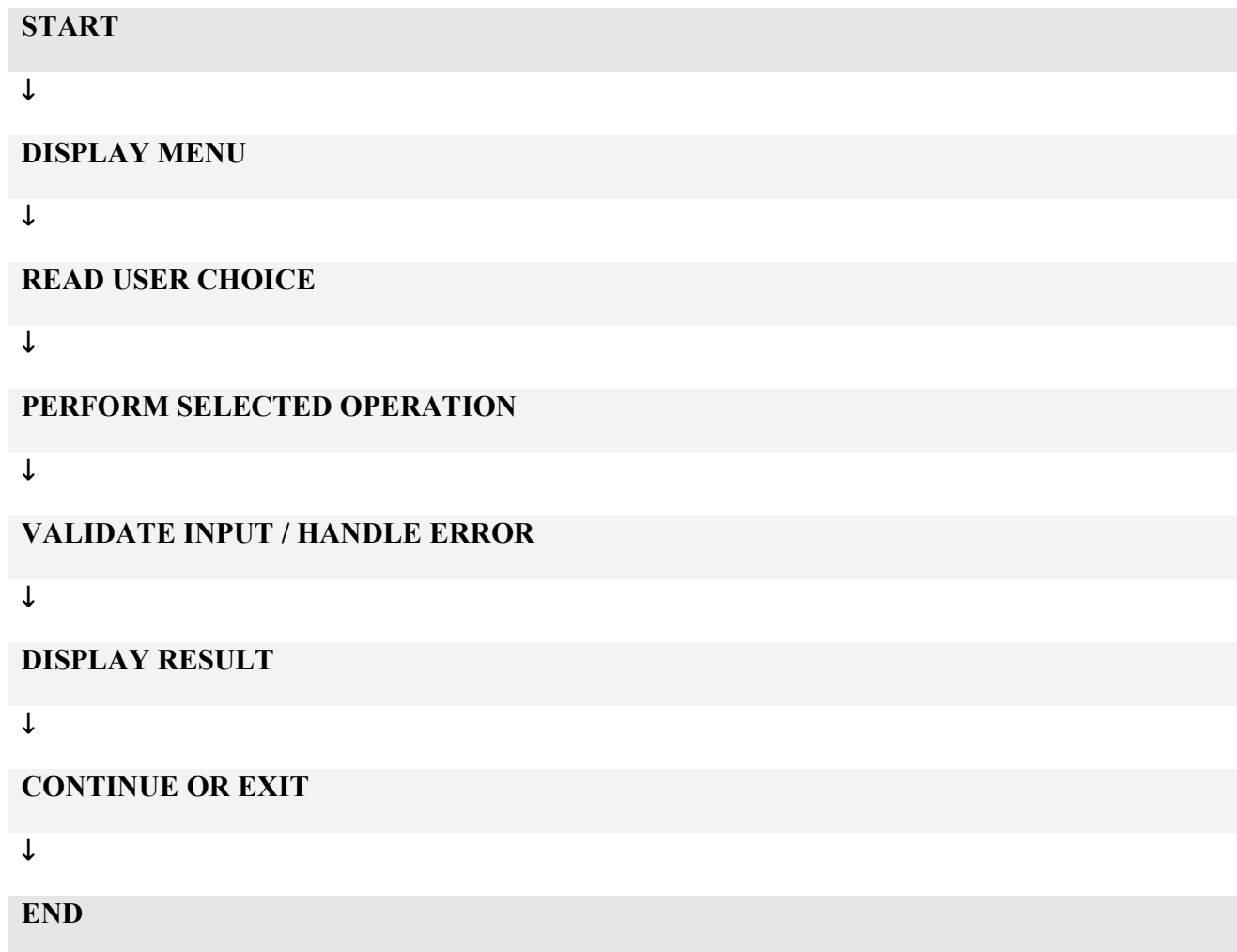
1.1 Objective

To build a simple interactive calculator that performs basic arithmetic operations through a menu-driven interface. The project focuses on functions, loops, conditional logic, input validation, exception handling, and logging.

1.2 Tools / Technologies

Python, functions, loops, conditionals, exception handling, logging

1.3 Design Flow



Mini Project 1: Menu-Driven Calculator

1.4 Learning

- Designed a menu-based program for repeated user interaction.
- Separated arithmetic operations into functions for better reuse.
- Used loops and conditional statements to control the menu.
- Validated user input and handled invalid entries safely.
- Prevented division by zero and other runtime failures.
- Used logging to record important calculator operations.
- Improved code readability through modular organization.

1.5 Conclusion

The calculator project provided a practical way to combine core Python concepts into one interactive program. It helped strengthen problem-solving, input validation, exception handling, and modular programming.

Mini Project 2: Transaction Management System

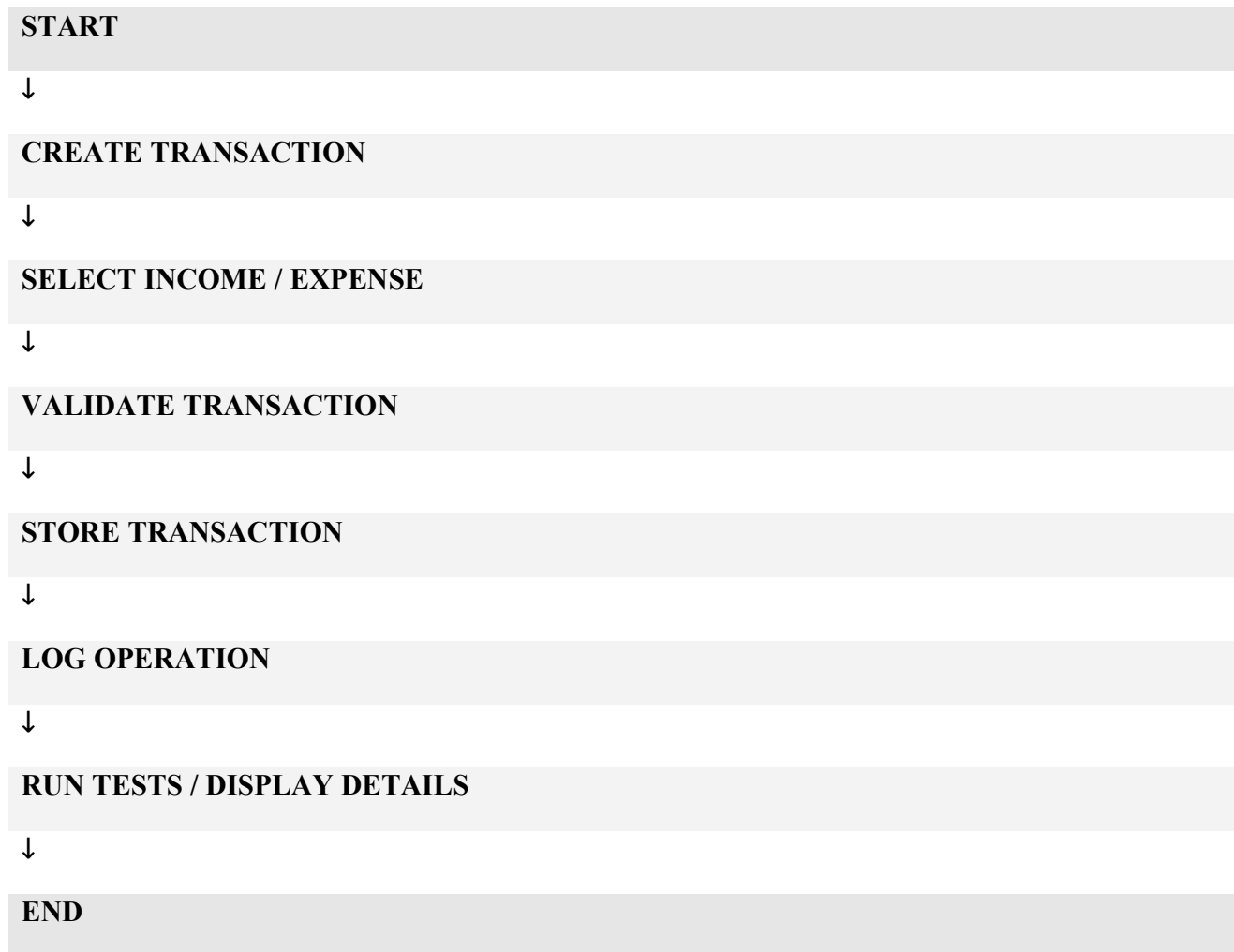
2.1 Objective

To develop a transaction management system using Object-Oriented Programming for handling income and expense records. The project applies inheritance, validation, exception handling, logging, and unit testing.

2.2 Tools / Technologies

Python, OOP, inheritance, logging, exception handling, unittest, mock testing

2.3 Design Flow



Mini Project 2: Transaction Management System

2.4 Learning

- Created separate classes to represent different types of transactions.
- Applied inheritance to reuse common transaction behaviour.
- Used constructors to initialize transaction objects.
- Validated amounts and categories before accepting transactions.
- Used custom error handling for invalid transaction data.
- Recorded transaction information and errors using logging.
- Used unit tests and mock testing to verify program behaviour.

2.5 Conclusion

This project demonstrated how OOP can be combined with validation and testing to build a more reliable application. It improved understanding of class design, inheritance, logging, and software testing.

Mini Project 3: Titanic Dataset Analysis

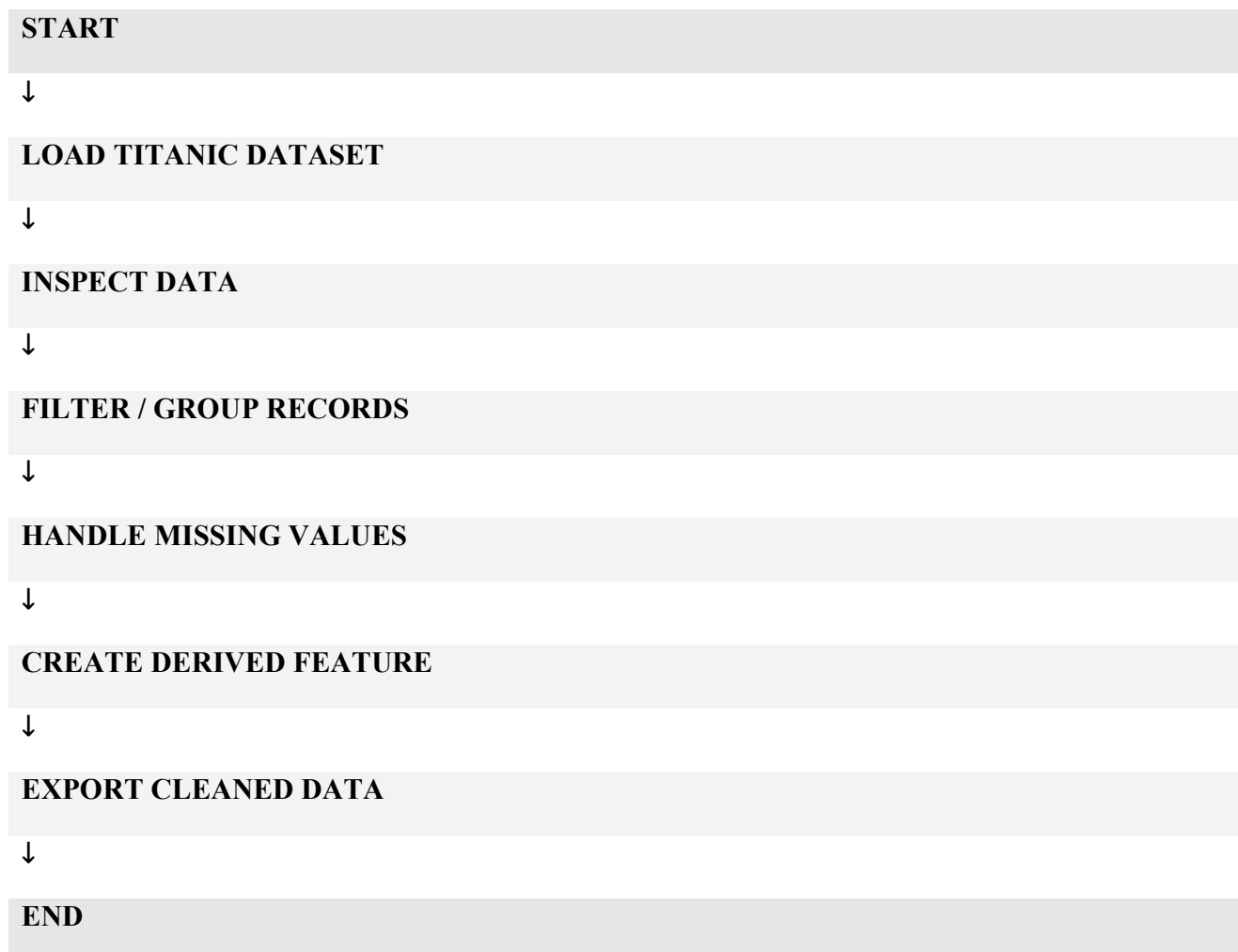
3.1 Objective

To perform exploratory analysis on the Titanic dataset using Pandas. The project focuses on loading data, filtering records, grouping passengers, handling missing values, creating derived features, and exporting processed data.

3.2 Tools / Technologies

Python, Pandas, CSV dataset, DataFrame operations

3.3 Design Flow



Mini Project 3: Titanic Dataset Analysis

3.4 Learning

- Loaded a CSV dataset into a Pandas DataFrame.
- Inspected and selected relevant columns and records.
- Used Boolean filtering to answer specific data questions.
- Grouped passenger information using `groupby()`.
- Performed aggregation to compare passenger groups.
- Created a new categorical feature using `pd.cut()`.
- Handled missing values using `fillna()`.
- Exported the processed dataset as a new CSV file.

3.5 Conclusion

The project introduced a practical workflow for exploratory data analysis. It showed how raw tabular data can be inspected, filtered, transformed, and exported using Pandas.

Mini Project 4: Data Analysis and Feature Engineering

4.1 Objective

To perform further preprocessing and feature engineering on the Titanic dataset. The project applies grouping, normalization, missing-value treatment, conditional feature creation, and pivot-table analysis.

4.2 Tools / Technologies

Python, Pandas, Min-Max normalization, Z-score normalization, pivot tables

4.3 Design Flow



Mini Project 4: Data Analysis and Feature Engineering

4.4 Learning

- Performed grouped analysis and aggregation using Pandas.
- Applied Min-Max normalization to scale numerical values.
- Used Z-score normalization to standardize features.
- Handled missing numerical values using suitable replacement values.
- Created new categorical features using conditional logic.
- Built pivot tables to compare multiple dimensions of the dataset.
- Understood the role of preprocessing before machine-learning tasks.
- Improved practical data-cleaning and feature-engineering skills.

4.5 Conclusion

This project extended the earlier dataset analysis into more structured preprocessing. It strengthened understanding of normalization, feature engineering, grouping, and pivot-table analysis as preparation for machine-learning workflows.

Mini Project 5: Simple To-Do Web Application

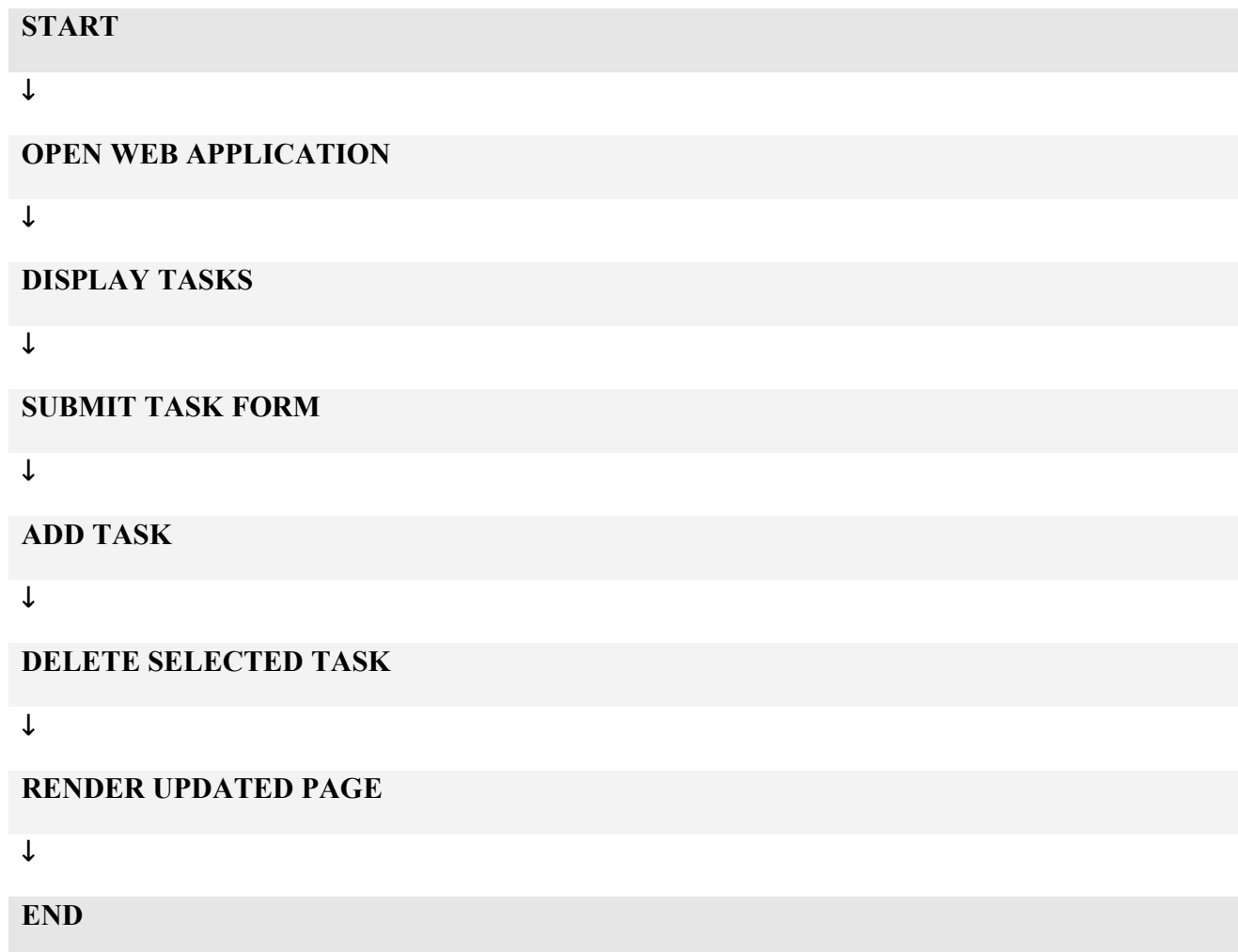
5.1 Objective

To develop a simple web-based To-Do application using Flask that allows users to add, view, and delete tasks. The project introduces routing, HTML templates, forms, and basic application-state management without a database.

5.2 Tools / Technologies

Python, Flask, HTML, Jinja2 templates, CSS

5.3 Design Flow



Mini Project 5: Simple To-Do Web Application

5.4 Learning

- Created a basic web application using Flask.
- Implemented routes for different application actions.
- Used Jinja2 templates to generate dynamic HTML.
- Handled user input through POST form requests.
- Maintained the task list using Python application state.
- Implemented add, view, and delete task operations.
- Connected the Python backend with a simple user interface.

5.5 Conclusion

The To-Do application provided hands-on experience with the basic structure of a Flask web application. It demonstrated how routes, forms, templates, and Python logic work together to create an interactive web page.

Mini Project 6: MQTT Publisher and Subscriber

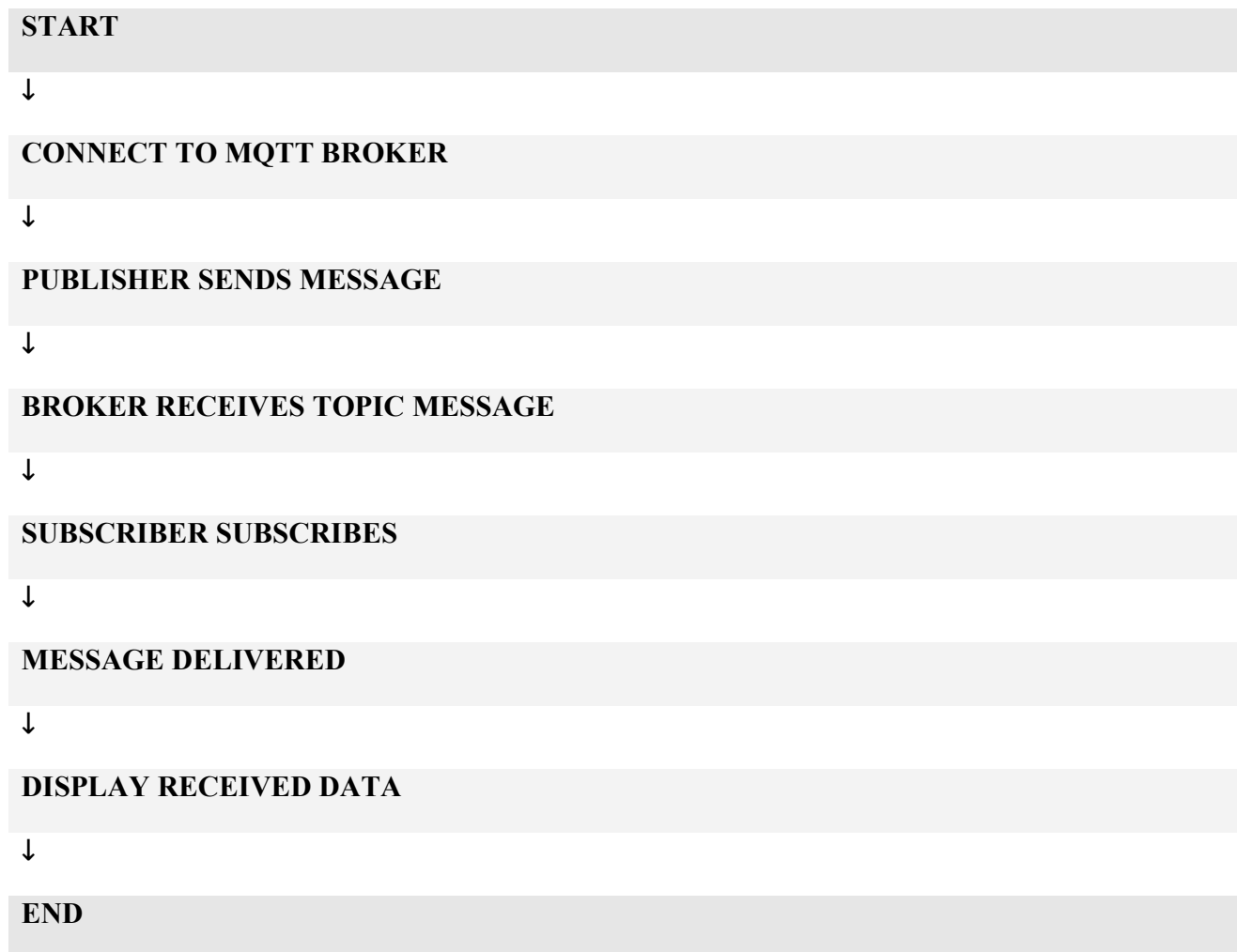
6.1 Objective

To implement real-time message communication using MQTT and the Paho-MQTT library. The project consists of a publisher that sends messages to an MQTT topic and a subscriber that receives messages through an MQTT broker.

6.2 Tools / Technologies

Python, Paho-MQTT, MQTT broker, publisher/subscriber model

6.3 Design Flow



Mini Project 6: MQTT Publisher and Subscriber

6.4 Learning

- Understood the MQTT publish-subscribe communication model.
- Learned how an MQTT broker connects publishers and subscribers.
- Implemented separate publisher and subscriber programs.
- Published messages to an MQTT topic.
- Subscribed to the same topic and processed received messages.
- Understood the role of callbacks in receiving messages.
- Gained practical exposure to asynchronous communication used in IoT.

6.5 Conclusion

The MQTT project demonstrated how lightweight messaging can be used for communication between IoT components. It provided practical understanding of publishers, subscribers, brokers, topics, and real-time message delivery.

CONCLUSION

The ECRIO work provided a progression from basic Python programming to practical application development. The exercises established the programming concepts, while the mini projects gave an opportunity to combine those concepts into complete, understandable applications. The projects covered problem solving, object-oriented design, data analysis, web development and IoT communication. Overall, the work improved my ability to choose suitable Python tools, structure programs, validate input, work with data, and explain how different components interact.